Project 2: Modelling daily cycling demand in Edinburgh

Portfolio version: Yixin Sun and project teammates (student IDs removed for privacy)

Word count: [2541 words]

1 Introduction

This report analyses daily cycling demand in Edinburgh using automatic counter data from 2020 to 2025. The goal is to develop a sequence of linear regression models (M0–M3) that capture the influence of weather, seasonality, day of week, and long term trends. Reliable demand models help transport planners allocate infrastructure, schedule maintenance, and promote active travel. By evaluating these models with cross-validation, we identify the best performing specifications and use it to answer practical planning problems.

2 Exploratory data analysis

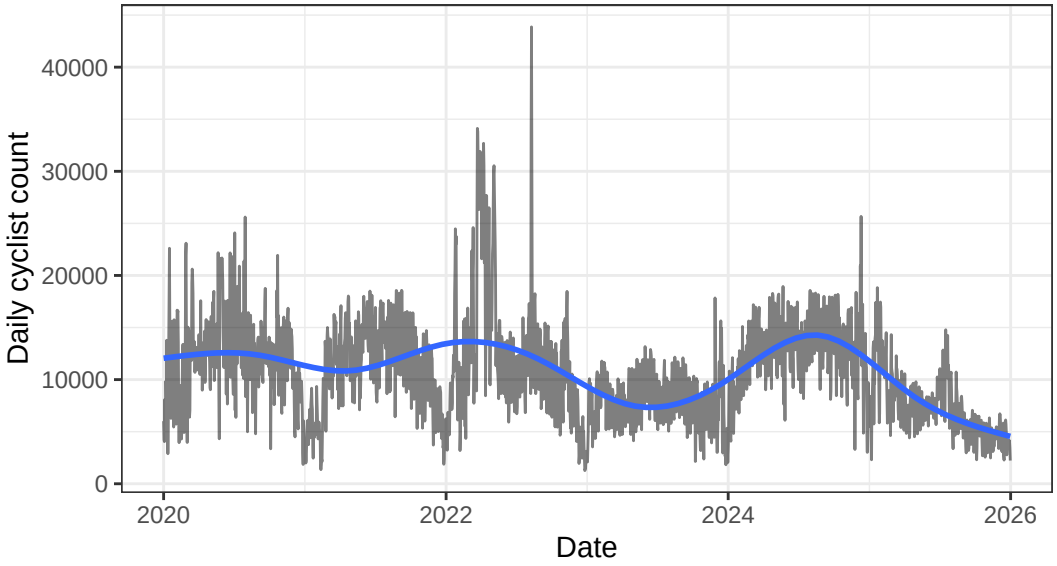
The exploratory analysis was used to describe the distribution of daily cyclist demand and to identify features relevant for later model development. For this purpose, `month` was converted to an ordered factor to preserve calendar order in plots and regression terms, while `dow` was explicitly releveled from Sunday to Saturday to retain its natural weekly sequence. A linear trend variable was also defined as the number of days since 1 January 2020, providing an interpretable measure of long run temporal change.

Table 1: Summary statistics for daily cyclist count and temperature variables.

Statistic	count	temp_mean	temp_min	temp_max
N	2180.00	2180.00	2180.00	2180.00
Mean	10815.47	9.69	5.92	13.15
SD	4728.88	4.79	4.98	5.31
Min	1264.00	-8.08	-14.00	-3.00
Q1	7476.00	6.25	2.00	9.00
Median	10531.00	9.82	6.00	13.00
Q3	13720.00	13.54	10.00	17.00
Max	43864.00	22.96	18.00	31.00

The response variable exhibits substantial variability over the study period. From Table 1, daily cyclist counts have a mean of 10,815 and a median of 10,531, with values ranging from 1,264 to 43,864, indicating a broad spread and the presence of some unusually high demand days. Mean daily temperature averages 9.69°C, with an interquartile range from 6.25°C to 13.54°C, indicating sufficient variation for assessing weather effects.

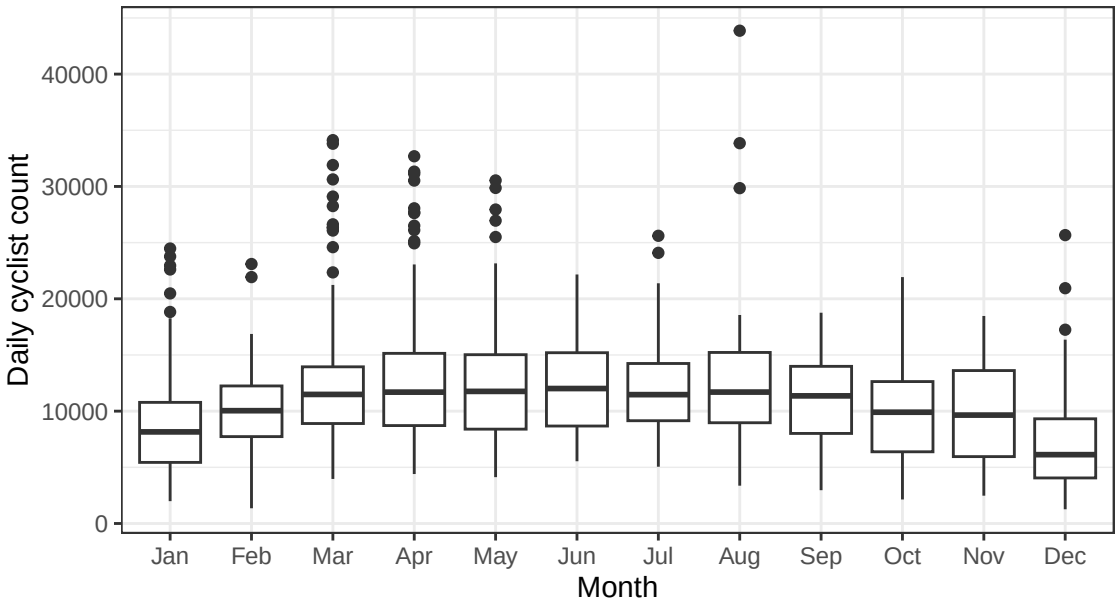
Figure 1: Daily cyclist counts in Edinburgh, 2020−−2025



Daily cyclist counts over time with a smooth trend highlighting long term changes.

Figure 1 displays the daily cyclist count series from 2020 to 2025. A pronounced seasonal cycle is evident, with lower counts in winter and higher counts during warmer months. In addition, the overall level of the series changes over time rather than remaining constant, indicating longer run drift in cyclist demand. The earliest period appears irregular, which is consistent with potential COVID period disruption, although this pattern alone does not establish causality. Overall, the time series display supports the later inclusion of a trend term in addition to seasonal covariates.

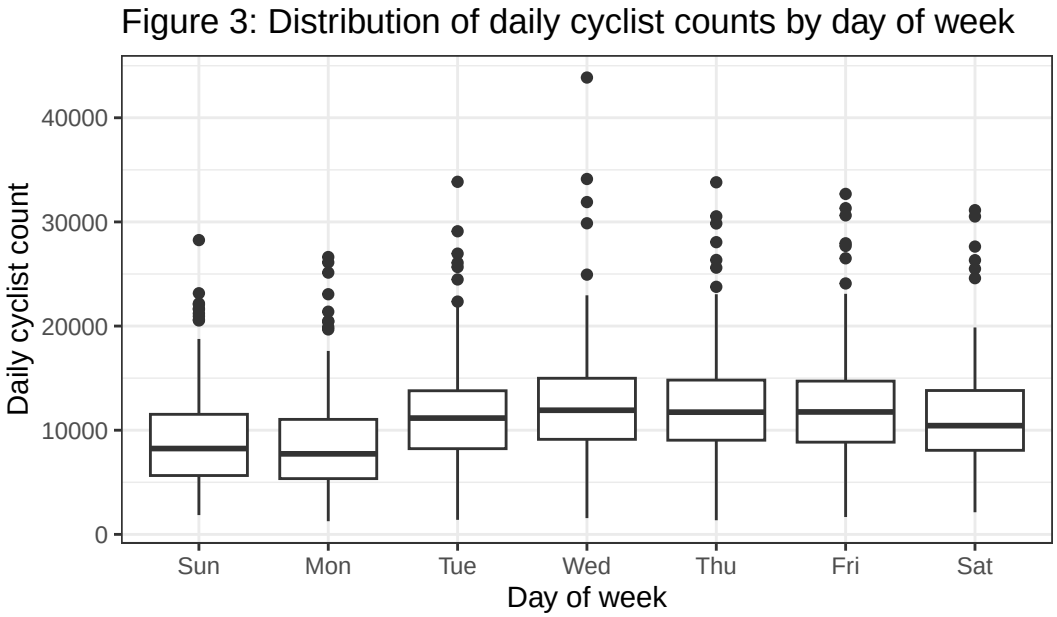
Figure 2: Distribution of daily cyclist counts by month



Boxplots of daily cyclist counts by month, showing seasonal differences in central tendency and variability.

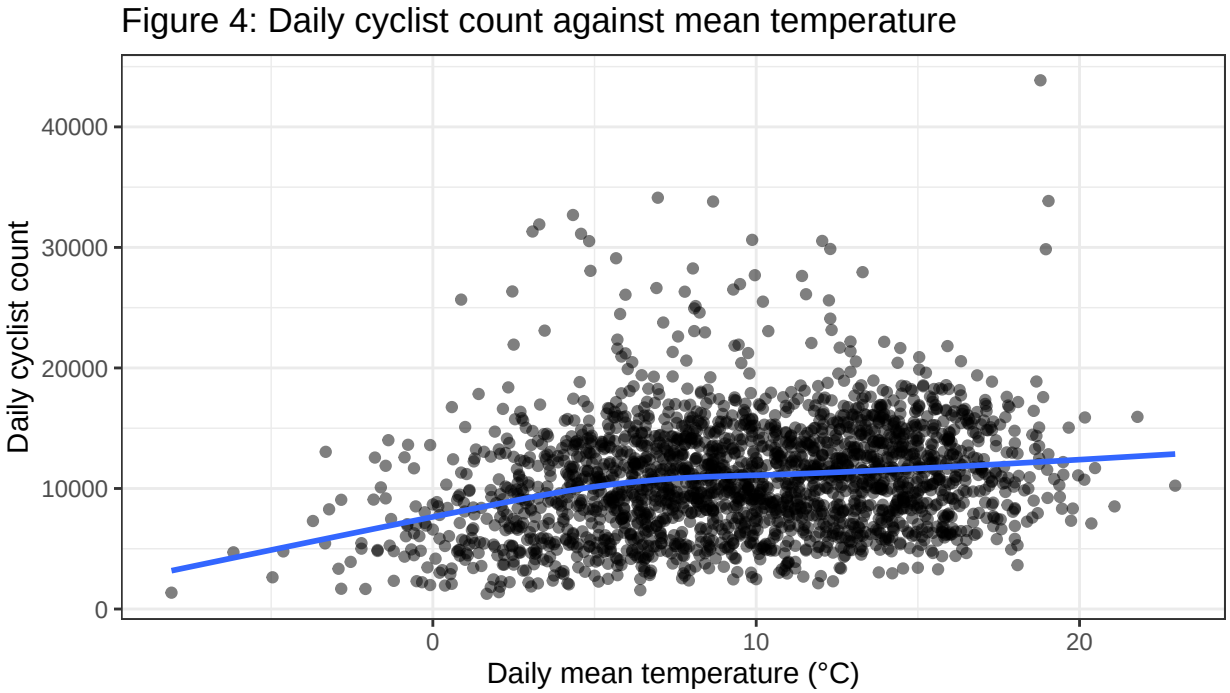
Figure 2 further supports the presence of seasonality. Median cyclist counts are lowest in winter, especially in December and January, and generally higher from spring to early autumn. Several warmer months exhibit greater variability and more extreme upper tail observations. Thus, it is implied that monthly effects should

be represented flexibly in later models.



Boxplots of daily cyclist counts by day of week, showing systematic within−week variation.

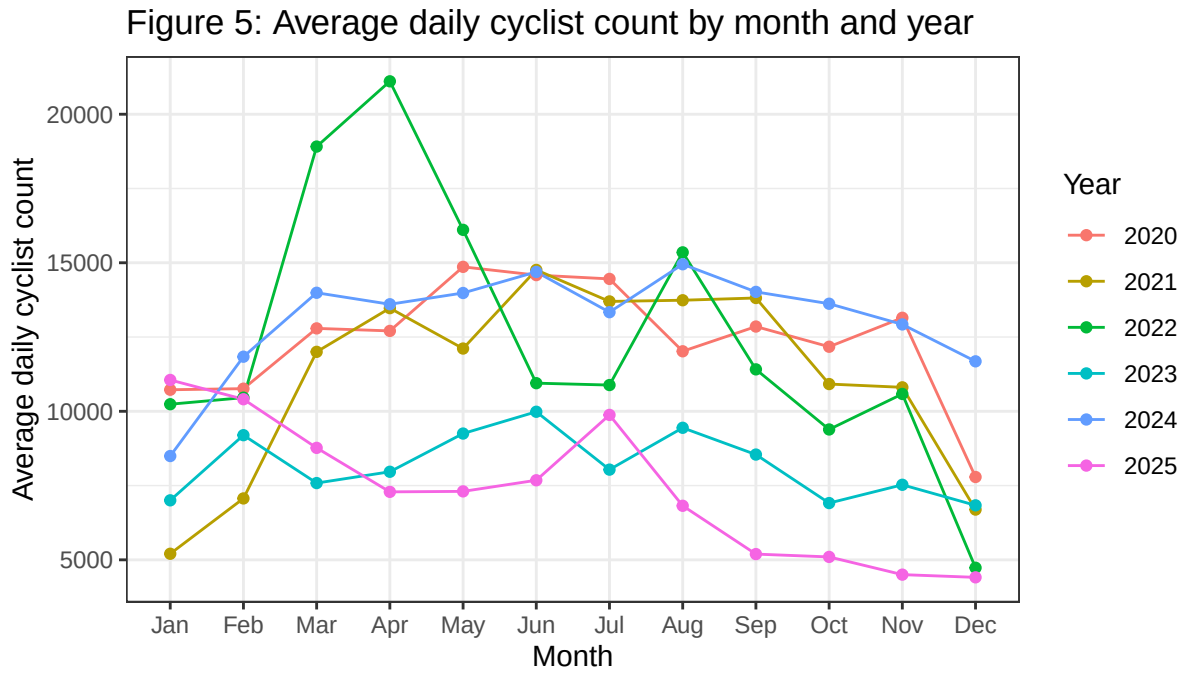
Figure 3 shows a systematic within-week pattern. Median cyclist counts are lowest on Sunday and Monday, are noticeably higher from Tuesday to Friday, and remain intermediate on Saturday. This structure is consistent with a mixture of commuting and leisure cycling demand, with weekday travel appearing particularly important. The plot therefore suggests that modelling only a binary weekend indicator may be too restrictive, and it provides direct support for including day-of-week effects more flexibly in later models.



Scatter plot of cyclist count against mean temperature, to assess whether the temperature effect appears nonlinear.

Figure 4 shows a positive association between cyclist count and mean temperature. However, the smooth

curve suggests that the relationship may not be strictly linear, with a steeper increase at lower temperatures and some flattening at higher values. Temperature is therefore likely to matter, but a nonlinear specification may be preferable.



Monthly mean cyclist counts by year, showing seasonality and possible COVID period disruption.

Figure 5 compares monthly mean counts across years and shows that seasonal profiles differ between years. In particular, 2020-2022 appear less regular than later years, again suggesting possible COVID period disruption. This additional plot reinforces the idea that both seasonality and longer run temporal change are important, and it suggests that year specific disruption may need to be revisited later if diagnostic plots indicate residual temporal structure.

Therefore, the exploratory analysis indicates strong seasonality, systematic day-of-week effects, a positive but potentially nonlinear temperature effect, and longer run temporal change, all of which motivate the richer models considered in the following sections.

3 Baseline model M0

The baseline model M0 is specified as:

$$\text{count}_i = \beta_0 + \beta_1 \text{temp\_mean}_i + \beta_2 \text{weekend}_i + \beta_3 \text{month}_i + \varepsilon_i,$$

where `month` is treated as a numeric variable. This provides a simple benchmark without day-of-week factors or a time trend.

Table 2: Coefficient table for M0.

term	estimate	std.error	statistic	p.value	conf.low	conf.high
Intercept	10385.819	267.015	38.896	0	9862.187	10909.451
temp_mean	247.179	20.769	11.901	0	206.450	287.908
weekend	-1246.888	214.172	-5.822	0	-1666.892	-826.885

month	-247.184	28.810	-8.580	0	-303.681	-190.687

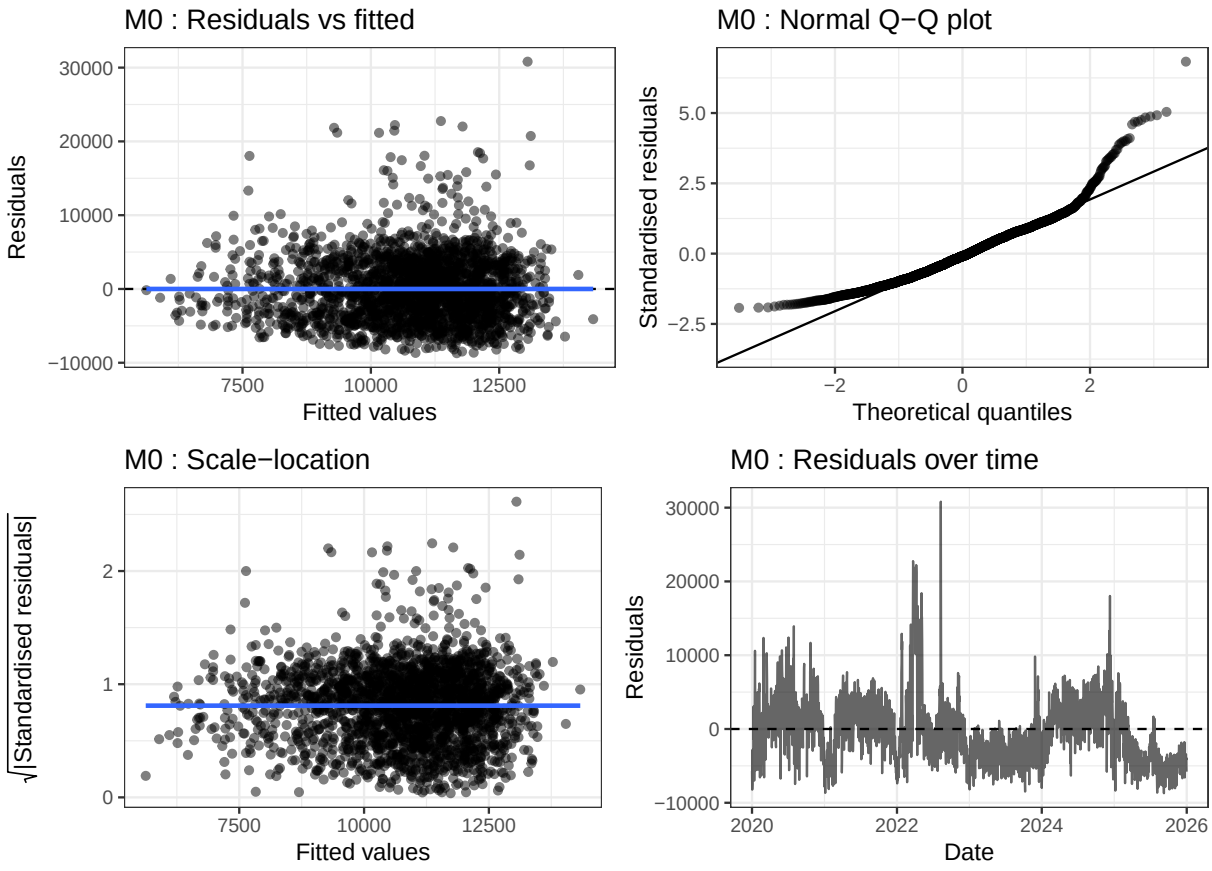


Figure 6: Diagnostic plots for M0.

M0 captures the broad seasonal pattern via the numeric month term, but the diagnostic plots reveal clear deficiencies. From Figure 6, the residuals vs fitted plot shows increasing spread with larger fitted values, indicating heteroscedasticity. Non- normality is indicated through heavy tails exhibited by the Q-Q plot. The residuals over time show a downward drift, suggesting a missing time trend. These issues motivate the inclusion of a linear trend and flexible day-of-week effects in M1.

4 Proposed models M1–M3

4.1 Model M1: Trend and factor terms

M1 extends M0 by adding a linear trend, replacing numeric `month` with month-specific intercepts, and including full day-of-week effects:

$$\text{count}_i = \beta_0 + \beta_1 \text{temp\_mean}_i + \beta_2 \text{weekend}_i + \beta_3 \text{trend}_i + \sum_{m=2}^{12} \gamma_m \mathbb{I}(\text{month}_i = m) + \sum_{d=2}^{7} \delta_d \mathbb{I}(\text{dow}_i = d) + \varepsilon_i$$

The trend term captures long-run changes visible in Figure 1. Using `factor(month)` allows each month to have its own intercept, addressing the nonlinear seasonal pattern seen in Figure 2. The day-of-week factors (with Sunday as baseline) account for the systematic weekly variation shown in Figure 3.

Table 3: Coefficient table for M1 (selected terms).

term	estimate	std.error	statistic	p.value	conf.low	conf.high
Intercept	10924.360	405.201	26.960	0.000	10129.735	11718.986
temp_mean	164.045	31.554	5.199	0.000	102.164	225.925
weekend	-2976.989	322.856	-9.221	0.000	-3610.131	-2343.848
trend	-1.895	0.139	-13.613	0.000	-2.167	-1.622
factor(month)2	1038.811	428.561	2.424	0.015	198.375	1879.246
factor(month)3	3341.690	422.200	7.915	0.000	2513.730	4169.650
factor(month)4	3466.538	435.083	7.968	0.000	2613.313	4319.763
factor(month)5	2576.147	471.321	5.466	0.000	1651.857	3500.437
factor(month)6	2045.081	520.053	3.932	0.000	1025.225	3064.937
factor(month)7	1427.334	545.460	2.617	0.009	357.652	2497.016
factor(month)8	1950.211	542.113	3.597	0.000	887.094	3013.329
factor(month)9	1200.972	501.863	2.393	0.017	216.786	2185.157
factor(month)10	440.311	461.035	0.955	0.340	-463.808	1344.429
factor(month)11	1252.490	432.072	2.899	0.004	405.169	2099.811
factor(month)12	-1294.669	421.197	-3.074	0.002	-2120.662	-468.675
dow_modelMon	-3432.671	322.866	-10.632	0.000	-4065.831	-2799.511
dow_modelTue	-681.358	322.874	-2.110	0.035	-1314.535	-48.181
dow_modelWed	252.200	322.610	0.782	0.434	-380.457	884.858
dow_modelThu	211.028	322.603	0.654	0.513	-421.617	843.673
dow_modelFri	NA	NA	NA	NA	NA	NA
dow_modelSat	2000.055	322.597	6.200	0.000	1367.422	2632.688

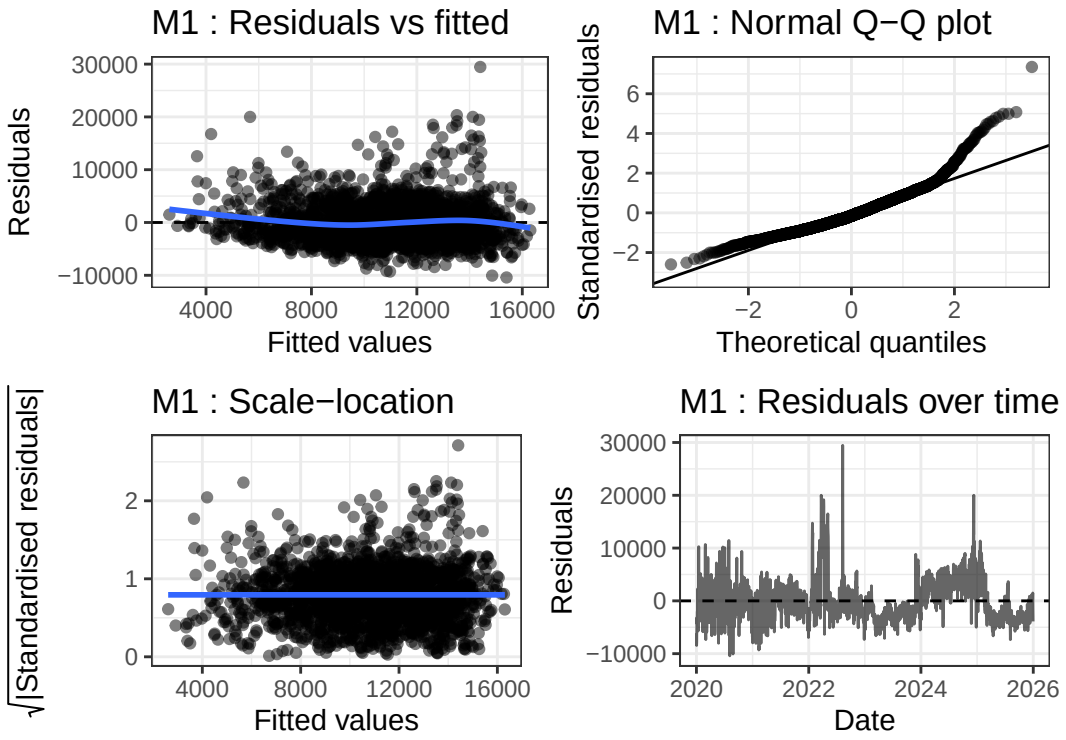


Figure 7: Diagnostic plots for M1.

Compared to M0, the residuals over time no longer show a strong drift, and the Q-Q plot is slightly improved but still shows right skew. The scale location plot indicates that heteroscedasticity remains. These observations lead to M2.

4.2 Model M2: Nonlinear temperature

M2 replaces the linear temperature term with a quadratic function to capture the curvature observed in Figure 4:

$$\text{count}_i = \beta_0 + \beta_1 \text{temp\_mean}_i + \beta_2 \text{temp\_mean}_i^2 + (\text{other terms as in M1}) + \varepsilon_i$$

where "other terms as in M1" refers to the linear trend, weekend indicator, month-specific intercepts, and day-of-week effects.

Table 4: Coefficient table for M2 (selected terms).

term	estimate	std.error	statistic	p.value	conf.low	conf.high
Intercept	10441.621	429.357	24.319	0.000	9599.626	11283.617
temp_mean	353.559	64.932	5.445	0.000	226.223	480.895
I(temp_mean^2)	-11.802	3.537	-3.337	0.001	-18.737	-4.866
weekend	-3005.389	322.214	-9.327	0.000	-3637.271	-2373.507
trend	-1.872	0.139	-13.470	0.000	-2.145	-1.600
factor(month)2	933.506	428.722	2.177	0.030	92.755	1774.258
factor(month)3	3165.501	424.509	7.457	0.000	2333.013	3997.989
factor(month)4	3248.318	438.964	7.400	0.000	2387.483	4109.153
factor(month)5	2480.247	471.096	5.265	0.000	1556.397	3404.097
factor(month)6	2238.225	522.055	4.287	0.000	1214.442	3262.008
factor(month)7	1840.913	558.119	3.298	0.001	746.406	2935.420
factor(month)8	2318.462	551.988	4.200	0.000	1235.978	3400.946
factor(month)9	1285.290	501.327	2.564	0.010	302.156	2268.425
factor(month)10	290.648	462.138	0.629	0.529	-615.634	1196.931
factor(month)11	1124.999	432.752	2.600	0.009	276.346	1973.653
factor(month)12	-1350.445	420.544	-3.211	0.001	-2175.159	-525.731
dow_modelMon	-3451.676	322.161	-10.714	0.000	-4083.454	-2819.897
dow_modelTue	-682.126	322.120	-2.118	0.034	-1313.822	-50.429
dow_modelWed	246.579	321.860	0.766	0.444	-384.608	877.767
dow_modelThu	221.396	321.864	0.688	0.492	-409.800	852.591
dow_modelFri	NA	NA	NA	NA	NA	NA
dow_modelSat	2002.362	321.844	6.222	0.000	1371.206	2633.518

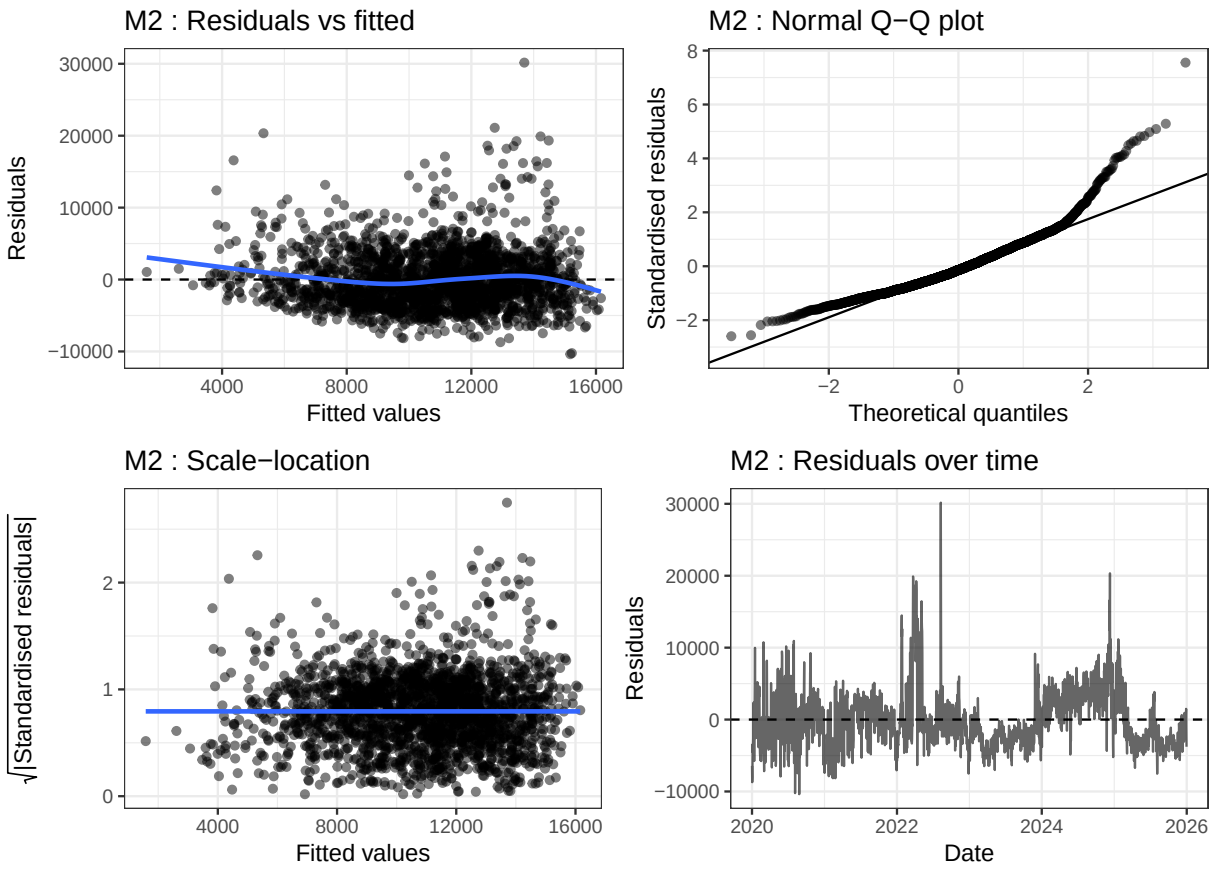


Figure 8: Diagnostic plots for M2.

The quadratic term is statistically significant and improves the fit marginally. However, the Q-Q plot still shows a heavy right tail, and the scale location plot continues to indicate nonconstant variance. This suggests that a transformation of the response or additional covariates may be needed.

4.3 Model M3: Extension (daily temperature range)

Our extension adds the daily temperature range (`temp_range = temp_max - temp_min`) as an additional predictor. The rationale is that the spread between minimum and maximum temperature may capture diurnal variability, which could affect cycling decisions independently of the mean temperature. For example, a large temperature range might indicate a rapid warm up or cool down, potentially discouraging cycling. The model is:

$$\begin{aligned}\text{count}_i =& \beta_0 + \beta_1 \text{temp\_mean}_i + \beta_2 \text{temp\_mean}_i^2 + \beta_3 \text{temp\_range}_i \\ &+ \beta_4 \text{weekend}_i + \beta_5 \text{trend}_i + \sum_{m=2}^{12} \gamma_m \mathbb{I}(\text{month}_i = m) \\ &+ \sum_{d=2}^{7} \delta_d \mathbb{I}(\text{dow}_i = d) + \varepsilon_i\end{aligned}$$

Table 5: Coefficient table for M3.

term	estimate	std.error	statistic	p.value	conf.low	conf.high
Intercept	9689.219	471.487	20.550	0.000	8764.603	10613.834
temp_mean	402.374	65.989	6.098	0.000	272.965	531.783
I(temp_mean^2)	-14.049	3.575	-3.930	0.000	-21.059	-7.038
weekend	-2978.765	321.289	-9.271	0.000	-3608.833	-2348.697
trend	-1.875	0.139	-13.533	0.000	-2.147	-1.604
factor(month)2	861.854	427.805	2.015	0.044	22.902	1700.806
factor(month)3	2861.528	430.663	6.644	0.000	2016.971	3706.086
factor(month)4	2756.339	456.300	6.041	0.000	1861.506	3651.171
factor(month)5	2007.411	485.793	4.132	0.000	1054.739	2960.082
factor(month)6	1857.715	529.952	3.505	0.000	818.445	2896.985
factor(month)7	1495.609	563.736	2.653	0.008	390.086	2601.132
factor(month)8	2001.691	556.534	3.597	0.000	910.292	3093.091
factor(month)9	906.554	509.584	1.779	0.075	-92.773	1905.880
factor(month)10	95.627	463.544	0.206	0.837	-813.412	1004.667
factor(month)11	980.380	433.078	2.264	0.024	131.086	1829.674
factor(month)12	-1361.101	419.247	-3.247	0.001	-2183.271	-538.931
dow_modelMon	-3432.555	321.199	-10.687	0.000	-4062.448	-2802.663
dow_modelTue	-674.440	321.125	-2.100	0.036	-1304.186	-44.693
dow_modelWed	241.227	320.863	0.752	0.452	-388.005	870.459
dow_modelThu	218.060	320.865	0.680	0.497	-411.177	847.297
dow_modelFri	NA	NA	NA	NA	NA	NA
dow_modelSat	1983.070	320.884	6.180	0.000	1353.796	2612.343
temp_range	110.281	28.981	3.805	0.000	53.447	167.116

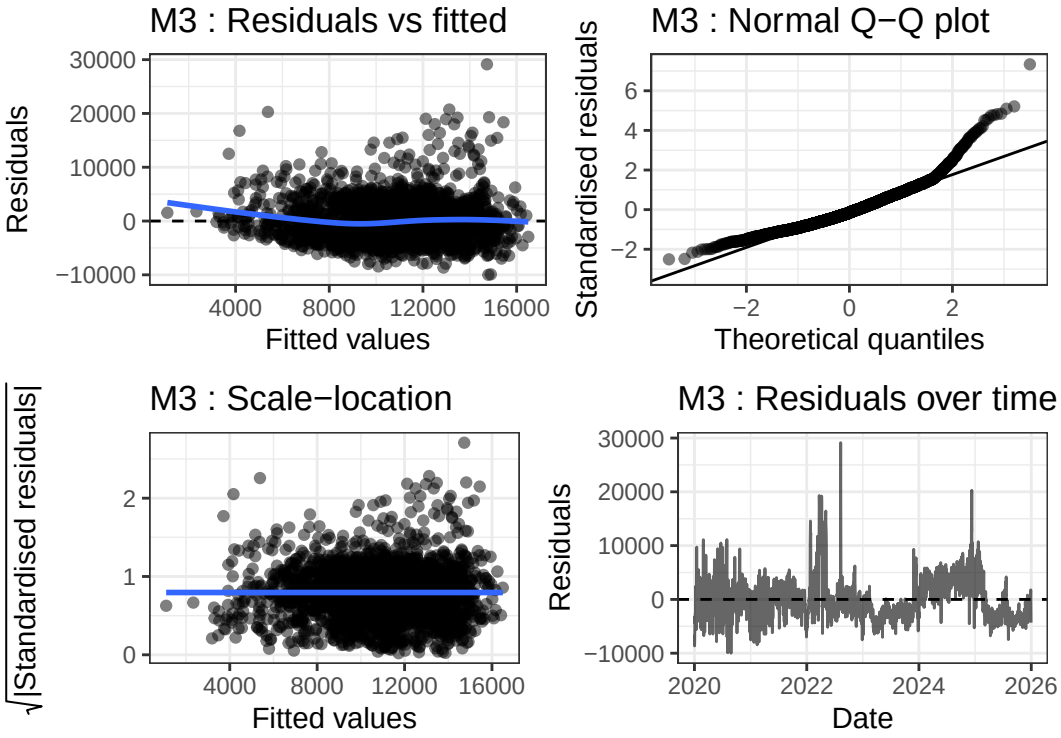


Figure 9: Diagnostic plots for M3.

The coefficient for `temp_range` is negative and statistically significant, indicating that larger diurnal temperature swings are associated with lower cycling demand, holding mean temperature constant. The diagnostic plots show modest improvement over M2, particularly in the scale-location plot, though the Q-Q plot remains somewhat heavy tailed.

5 Evaluation

5.1 Statistical Performance

5.2 Statistical Performance

We evaluate predictive performance using leave-one-year-out cross-validation (LOYO CV) for all four models. Table 6 reports the mean RMSE, MAE, DS and interval score (IS) across the six held out years (2020–2025). Lower values are better for all scores.

Table 6: Leave-one-year-out CV scores for M0–M3. Lower is better for all metrics.

Model	RMSE	MAE	DS	IS
M0	4740.9	3805.6	18.05	25630.5
M1	4604.5	3669.2	18.07	25856.4
M2	4585.5	3654.1	18.06	25647.4
M3	4585.7	3661.3	18.06	25370.2

For our final selected model (M2, with quadratic temperature and full factor terms), we further assess monthly predictive accuracy by holding out all observations from a given month (across all years) and training on the remaining 11 months. Table 7 shows the CV RMSE and DS for each month.

Table 7: CV RMSE and DS by month for the final model (M2).

Month	RMSE	DS
Jan	5197.2	18.25
Feb	3836.8	17.52
Mar	5447.3	18.44
Apr	5761.9	18.68
May	4323.3	17.75
Jun	3118.9	17.24
Jul	3016.6	17.21
Aug	4475.6	17.82
Sep	3236.1	17.28
Oct	3366.1	17.33
Nov	3714.6	17.47
Dec	4596.3	17.89

5.2.1 Disagreement among scores and final model choice

The four scores capture different aspects of predictive performance:

- **RMSE** (root mean squared error) heavily penalises large forecast errors due to the squaring operation. It is most relevant when large deviations are disproportionately costly (e.g., underestimating demand on a very busy day could lead to infrastructure strain).
- **MAE** (mean absolute error) treats all absolute errors linearly, making it more robust to outliers than RMSE. It reflects the typical absolute error in the original unit (cyclists/day).
- **Dawid–Sebastiani (DS) score** is a proper scoring rule that evaluates both the accuracy of the mean forecast and the calibration of the predictive variance. A lower DS indicates that the predicted uncertainty ( ) is well matched to the actual forecast errors.
- **Interval score (IS)** assesses the quality of 95% prediction intervals, rewarding narrow intervals that achieve the nominal coverage. It penalises intervals that are too wide or that miss the true value.

Because RMSE and MAE only evaluate point forecasts, while DS and IS also evaluate probabilistic calibration, they may disagree. For example, a model that correctly estimates the mean but underestimates variance will have good RMSE/MAE but poor DS and IS. Conversely, a model with slightly higher RMSE but better calibrated prediction intervals could be preferred for decision-making under uncertainty.

From Table 6, M2 and M3 perform similarly. M2 has marginally lower RMSE (4585.5 vs 4585.7) and MAE (3654.1 vs 3661.3), while M3 achieves a better interval score (IS: 25370.2 vs 25647.4). The DS scores are identical (18.06). Since the point forecast metrics (RMSE and MAE) are slightly better for M2 and the model is simpler (excluding temp_range), we select M2 as the final model for interpretability and operational planning.

5.3 Practical Performance

5.3.1 Temperature effects and nonlinearity

To assess which temperature measure is most informative, M2 was re-fitted three times using `temp_mean`, `temp_min`, and `temp_max` as the basis for the quadratic temperature term. Among the three specifications, the model based on `temp_max` performed best on all cross-validation criteria, with the lowest RMSE (4557.0), Dawid-Sebastiani score (18.05), and interval score (25340.1). We therefore retain `temp_max` as the preferred temperature variable for this planning analysis.

Table 8: Cross-validation comparison of M2 fitted with alternative temperature variables.

Temperature	RMSE	DS	IS
temp_mean	4585.5	18.06	25647.4
temp_min	4635.8	18.07	25766.5
temp_max	4557.0	18.05	25340.1

Table 9: Estimated marginal temperature effect for the selected M2 variant.

Evaluation point	Marginal effect (cyclists per 1°C)
5°C	314.1
15°C	141.4

This result is realistically plausible. Daily cycling decisions may depend more strongly on the warmest conditions experienced during the day than on the daily mean or minimum temperature, particularly for discretionary or recreational trips. The estimated marginal effects also support a nonlinear relationship. At 5°C, a 1°C increase in temperature is associated with approximately 314.1 additional cyclists per day,

whereas at 15°C the corresponding increase is 141.4 cyclists per day. Thus, temperature has a positive but diminishing effect on cycling demand, implying modest warming matters more on relatively cold days than on already mild days.

5.3.2 Seasonal prediction accuracy

From Table 7 in Section 5.1, the final model M2 performs best in July, with RMSE 3016.6 and DS 17.21, and worst in April, with RMSE 5761.9 and DS 18.68. This indicates that predictive accuracy varies materially across the calendar year. A plausible transport planning interpretation is that midsummer demand is relatively stable and therefore easier to capture using temperature, day-of-week, month, and trend effects. By contrast, April may be harder to predict because cycling demand is more volatile during the transition into warmer conditions and may also be influenced by omitted factors such as holidays, special events, or short term disruptions. This pattern should therefore be interpreted as reflecting both a model limitation and a data limitation, rather than a purely seasonal behavioural effect.

5.3.3 Long-term trend and extrapolation

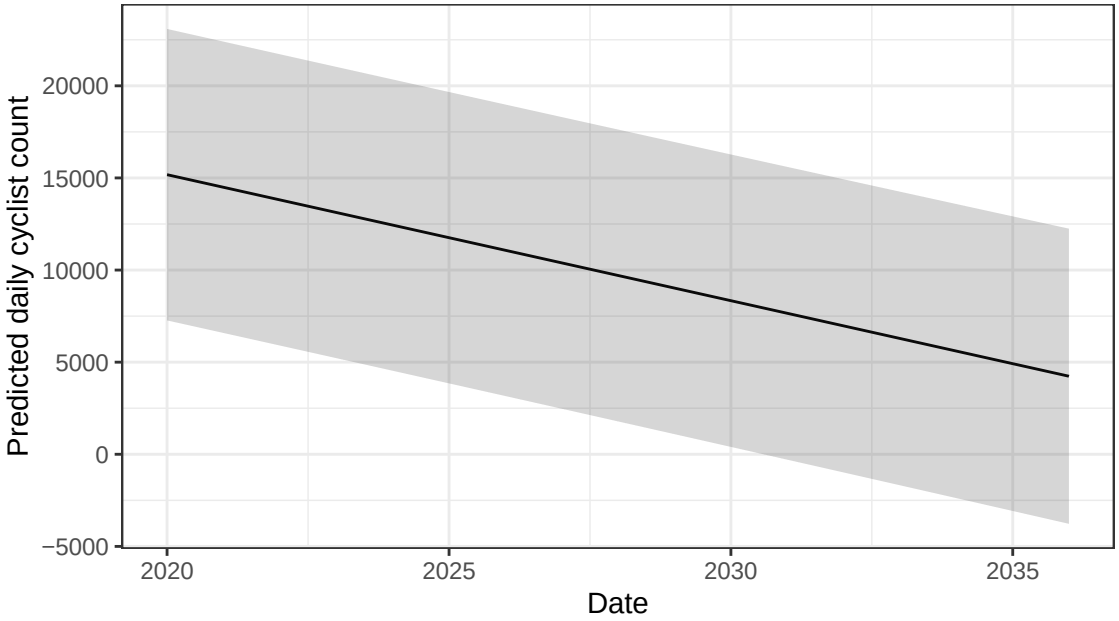
Table 10: Trend coefficients from M1–M3.

Model	Daily change (cyclists/day)	Annual change (cyclists/year)
M1	-1.895	-692
M2	-1.872	-683
M3	-1.875	-684

Table 10 shows that the estimated time trend is negative in all three models and is highly stable across specifications. The estimated daily change is -1.895 cyclists per day in M1, -1.872 in M2, and -1.875 in M3, corresponding to annual changes of approximately -692, -683, and -684 cyclists per year respectively. This consistency indicates that the substantive conclusion about long run decline over the observed period is robust to the alternative modelling choices considered here.

For the final model M2, the trend coefficient is -1.872 (95% CI: -2.145 to -1.600, $p < 0.001$), so the downward trend is statistically significant. Relative to the predicted count at 1 January 2020 (12,462.5 cyclists per day), this implies an annual percentage change of approximately -5.48%. In practical terms, the fitted model suggests that mean daily cycling demand declined over the observation window after controlling for temperature, day-of-week effects, and seasonality.

Figure 10: Projected daily cycling demand to 2035



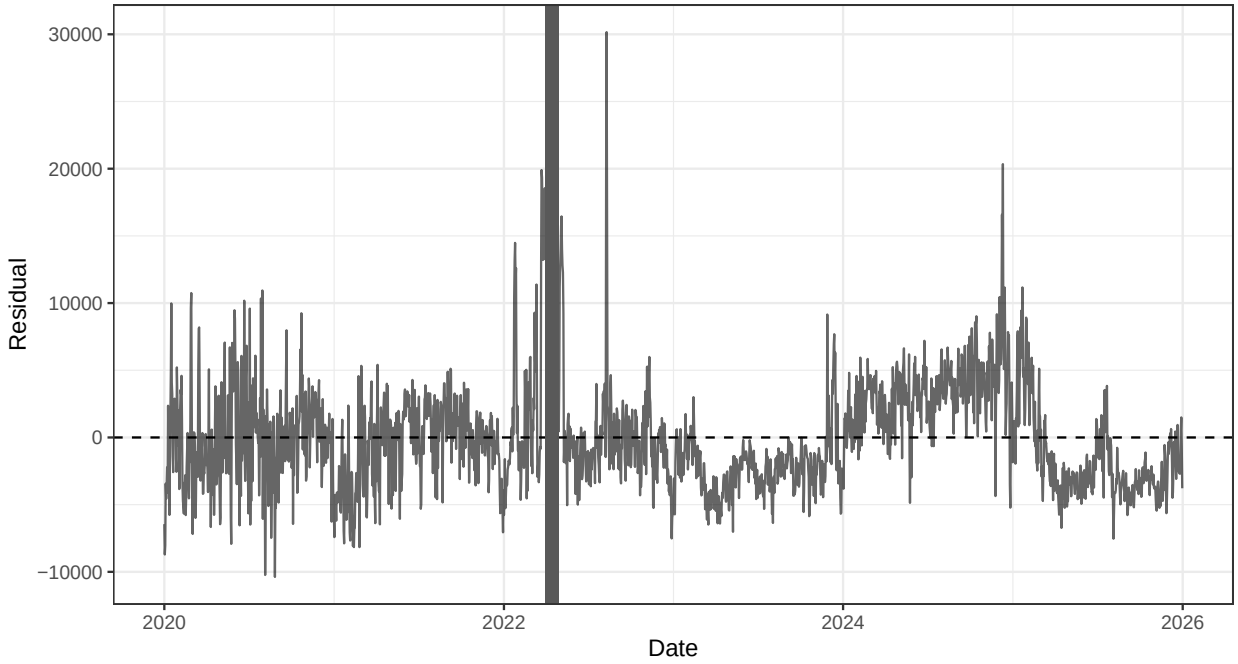
The projection under fixed summer weekday conditions extends this fitted trend to 2035. In this scenario, seasonality and weekday effects are held constant at a representative summer weekday profile, and temperature is fixed at a typical summer level. Because the estimated trend is negative, the projected series declines steadily throughout the horizon. Using a planning threshold of 10,000 cyclists per day, the model suggests that this level would be reached on 28 July 2027. This threshold is preferred here because higher thresholds such as 15,000 are crossed almost immediately(~4 months) and therefore provide little long run planning insight.

The COVID dip visible in 2020–2021 suggests that a single linear trend is only an approximation across the full sample period. For that reason, the negative trend should not be interpreted as a fully stable structural growth rate. More broadly, this extrapolation should be interpreted with caution for at least two reasons. First, it assumes that the linear trend estimated from 2020–2025 remains valid for another decade, despite the visible disruption associated with the pandemic period. Second, the model does not account for future infrastructure expansion, policy changes, behavioural adaptation, or other omitted drivers of cycling demand. The projection is therefore best viewed as a simple trend-based scenario rather than a realistic long-term forecast.

5.3.4 Identifying a poorly-fit period

Residuals over time indicate that the poorest fitting period for the final model is 2022-04. In this month, the mean residual is 8033.6 and the RMSE is 10528.8. Because the mean residual is positive, the model systematically underpredicts cycling demand during this period. A plausible explanation is that this month reflects a temporary surge in cycling activity not fully captured by the variables included in M2. Although the model adjusts for temperature, day-of-week effects, month effects, and a linear time trend, it does not include other short-term influences such as unusual weather patterns, holiday activity, special events, or post-pandemic behavioural shifts. From a transport planning perspective, this suggests that forecast errors may become especially large during periods of abrupt demand change.

Figure 11: Residuals over time for the final model (M2)

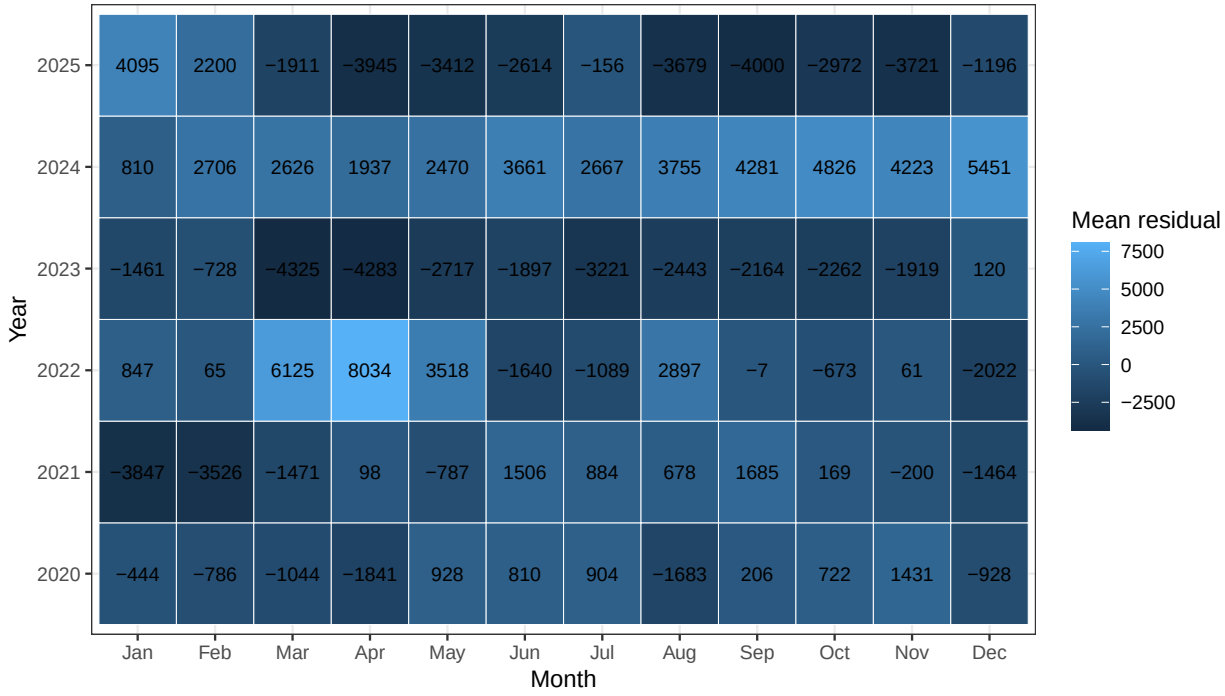


Residuals over time for M2, 2022−04 highlighted as the worst fit month. Positive residuals indicate under prediction.

5.3.5 Temporal patterns in forecast bias

We construct a heatmap of mean residuals by month and year for the final model M2 in order to examine whether prediction errors are randomly distributed over time. The plot shows clear temporal structure in model bias. Most months in 2023 have negative mean residuals, indicating systematic overprediction, whereas most months in 2024 have positive mean residuals, indicating systematic underprediction. Spring 2022, especially March and April, also stands out as a period of substantial underprediction, consistent with the poorly fitted period identified in Section 5.4. For transport planners, the main implication is that model reliability is not constant across the calendar. Forecasts may therefore require greater caution and wider operational margins in periods where the model shows repeated directional bias.

Figure 12: Systematic prediction bias by month and year for the final model (M2)



Mean residuals by month and year for the final model. Forecast bias is not uniform over time.

6 Discussion and limitations

The final model captures the main features of cycling demand in Edinburgh, including seasonality, day-of-week effects, longer run change, and nonlinear temperature effects. It improves meaningfully on the baseline model, but the diagnostics suggest that the linear model assumptions are only approximately satisfied.

Looking at the model's limitations, ordinary linear regression is being applied to a non-negative count outcome, so the model is best viewed as a practical approximation rather than a fully realistic model for cyclist demand. Next, the dataset omits potentially important drivers such as rainfall, wind, daylight hours, holidays, major events, infrastructure changes, and station availability, which likely contributes to uneven predictive performance across months and periods. Moreover, although the estimated trend is stable across models, the visible disruption in 2020–2021 means that a single linear trend over the full sample should be interpreted cautiously. For this reason, the projection to 2035 is better treated as a simple scenario under fixed conditions than as a literal long term forecast. Prediction could likely be improved by using a model better suited to count data, adding richer weather and disruption variables, and allowing a more flexible time trend.

Overall, the findings should be interpreted as specific to Edinburgh over 2020–2025 rather than assumed to generalise directly to other cities or future periods.

7 Appendix: Code

All analysis code is in `code.R` and is reproduced below.

```
# ============================================================
# Project 2: Modelling daily cycling demand in Edinburgh
```

```r
# # Authors: Yixin Sun and project teammates
# ============================================================

# 0. Packages
library(ggplot2); library(dplyr); library(tidyr)
library(lubridate); library(knitr); library(kableExtra)
library(patchwork); library(broom)

# 1. Load data
load('cycle_daily_df.Rdata')

# 2. Data Wrangling
cycle_daily_df <- cycle_daily_df %>%
  mutate(
    # Task 1: month as ordered factor for plots/inference
    month_f = factor(
      month,
      levels = 1:12,
      labels = month.abb,
      ordered = TRUE
    ),
    # Task 2: dow with explicit levels
    dow_plot = factor(
      dow,
      levels = c("Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"),
      ordered = TRUE
    ),
    dow_model = factor(
      dow,
      levels = c("Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"),
      ordered = FALSE
    ),
    # Task 3: trend as integer days since 2020-01-01
    trend = as.integer(date - as.Date("2020-01-01")),
    # other directions for m3
    log_count = log(count + 1),
    is_covid = ifelse(year %in% c(2020, 2021), 1, 0),
    is_summer = ifelse(month %in% c(6, 7, 8), 1, 0),
    #final m3 selection
    temp_range = temp_max - temp_min
  )

# Summary statistics table for key continuous variables
eda_summary <- tibble(
  Statistic = c("N", "Mean", "SD", "Min", "Q1", "Median", "Q3", "Max"),
  count = c(
    sum(!is.na(cycle_daily_df$count)),
    mean(cycle_daily_df$count, na.rm = TRUE),
    sd(cycle_daily_df$count, na.rm = TRUE),
    min(cycle_daily_df$count, na.rm = TRUE),
    quantile(cycle_daily_df$count, 0.25, na.rm = TRUE),
    median(cycle_daily_df$count, na.rm = TRUE),
    quantile(cycle_daily_df$count, 0.75, na.rm = TRUE),
```

```
    max(cycle_daily_df$count, na.rm = TRUE)
  ),
  temp_mean = c(
    sum(!is.na(cycle_daily_df$temp_mean)),
    mean(cycle_daily_df$temp_mean, na.rm = TRUE),
    sd(cycle_daily_df$temp_mean, na.rm = TRUE),
    min(cycle_daily_df$temp_mean, na.rm = TRUE),
    quantile(cycle_daily_df$temp_mean, 0.25, na.rm = TRUE),
    median(cycle_daily_df$temp_mean, na.rm = TRUE),
    quantile(cycle_daily_df$temp_mean, 0.75, na.rm = TRUE),
    max(cycle_daily_df$temp_mean, na.rm = TRUE)
  ),
  temp_min = c(
    sum(!is.na(cycle_daily_df$temp_min)),
    mean(cycle_daily_df$temp_min, na.rm = TRUE),
    sd(cycle_daily_df$temp_min, na.rm = TRUE),
    min(cycle_daily_df$temp_min, na.rm = TRUE),
    quantile(cycle_daily_df$temp_min, 0.25, na.rm = TRUE),
    median(cycle_daily_df$temp_min, na.rm = TRUE),
    quantile(cycle_daily_df$temp_min, 0.75, na.rm = TRUE),
    max(cycle_daily_df$temp_min, na.rm = TRUE)
  ),
  temp_max = c(
    sum(!is.na(cycle_daily_df$temp_max)),
    mean(cycle_daily_df$temp_max, na.rm = TRUE),
    sd(cycle_daily_df$temp_max, na.rm = TRUE),
    min(cycle_daily_df$temp_max, na.rm = TRUE),
    quantile(cycle_daily_df$temp_max, 0.25, na.rm = TRUE),
    median(cycle_daily_df$temp_max, na.rm = TRUE),
    quantile(cycle_daily_df$temp_max, 0.75, na.rm = TRUE),
    max(cycle_daily_df$temp_max, na.rm = TRUE)
  )
)

# 2. Exploratory Data Analysis (EDA)
#summary statistics
eda_summary %>%
  kable(
    digits = 2,
    caption = "Summary statistics for daily cyclist count and temperature variables."
  ) %>%
  kable_styling(full_width = FALSE)
eda_summary

#time series plot
p_ts <- ggplot(cycle_daily_df, aes(x = date, y = count)) +
  geom_line(alpha = 0.5) +
  geom_smooth(se = FALSE) +
  theme_bw() +
  labs(
    title = "Figure 1: Daily cyclist counts in Edinburgh, 2020--2025",
    x = "Date",
    y = "Daily cyclist count",
```

```
    caption = "Daily cyclist counts over time with a smooth trend highlighting long term changes."
  )
p_ts

#boxplot by month
p_box_month <- ggplot(cycle_daily_df, aes(x = month_f, y = count)) +
  geom_boxplot() +
  theme_bw() +
  labs(
    title = "Figure 2: Distribution of daily cyclist counts by month",
    x = "Month",
    y = "Daily cyclist count",
    caption = "Boxplots of daily cyclist counts by month, showing seasonal differences in central tende
  )
p_box_month

#boxplot by day of week
p_box_dow <- ggplot(cycle_daily_df, aes(x = dow_plot, y = count)) +
  geom_boxplot() +
  theme_bw() +
  labs(
    title = "Figure 3: Distribution of daily cyclist counts by day of week",
    x = "Day of week",
    y = "Daily cyclist count",
    caption = "Boxplots of daily cyclist counts by day of week, showing systematic within-week variatio
  )
p_box_dow

#scatter plot of count vs temp_mean
p_scatter_temp <- ggplot(cycle_daily_df, aes(x = temp_mean, y = count)) +
  geom_point(alpha = 0.5) +
  geom_smooth(se = FALSE) +
  theme_bw() +
  labs(
    title = "Figure 4: Daily cyclist count against mean temperature",
    x = "Daily mean temperature (°C)",
    y = "Daily cyclist count",
    caption = "Scatter plot of cyclist count against mean temperature, to assess whether the temperatur
  )
p_scatter_temp

#average daily cyclist count for each month by year.
monthly_mean_df <- cycle_daily_df %>%
  group_by(year, month_f) %>%
  summarise(
    mean_count = mean(count, na.rm = TRUE),
    .groups = "drop"
  )

p_monthly_mean_year <- ggplot(monthly_mean_df, aes(x = month_f, y = mean_count, group = factor(year), c
  geom_line() +
  geom_point() +
  theme_bw() +
```

```
  labs(
    title = "Figure 5: Average daily cyclist count by month and year",
    x = "Month",
    y = "Average daily cyclist count",
    colour = "Year",
    caption = "Monthly mean cyclist counts by year, showing seasonality and possible COVID period disru
  )
p_monthly_mean_year

# 3. Model Fitting
#chekclist and navigation
# 3.1 Baseline Model (M0)
#   3.1.1 M0 fit
#   3.1.2 M0 coefficient table
#   3.1.3 M0 diagnostic plots
# 3.2 Proposed Models (M1-M3)
#   3.2.1 M1 fit
#   3.2.2 M2 fit
#   3.2.3 M3 fit (final chosen extension: daily temperature rnge)
#   3.2.4 coefficient tables
#   3.2.5 diagnostic plots
#   3.2.6 exploratory M3 variants tested but not selected


# 3.1.1 M0 fit
m0 <- lm(count ~ temp_mean + weekend + month, data = cycle_daily_df)

# 3.2.1 M1 fit
m1 <- lm(count ~ temp_mean + weekend + trend + factor(month) + dow_model,
         data = cycle_daily_df)

# 3.2.2 M2 fit
m2 <- lm(count ~ temp_mean + I(temp_mean^2) + weekend + trend +
           factor(month) + dow_model,
         data = cycle_daily_df)

# 3.2.3 M3 fit (final chosen extension: daily temperature rnge)
m3 <- lm(count ~ temp_mean + I(temp_mean^2) + weekend + trend +
           factor(month) + dow_model + temp_range,
         data = cycle_daily_df)

# 3.2.6 exploratory M3 variants (tested but not selected) log model
m3_log <- lm(log_count ~ temp_mean + I(temp_mean^2) + weekend + trend +
               factor(month) + dow_model,
             data = cycle_daily_df)

# 3.2.6 exploratory M3 variants (tested but not selected)COVID model
m3_covid <- lm(count ~ temp_mean + I(temp_mean^2) + weekend + trend +
                 factor(month) + dow_model + is_covid,
               data = cycle_daily_df)

# 3.2.6 exploratory M3 variants (tested but not selected)temperature × summer interaction
m3_interaction <- lm(count ~ temp_mean + I(temp_mean^2) + weekend + trend +
```

```
                      factor(month) + dow_model + temp_mean:is_summer,
                    data = cycle_daily_df)

# 3.2.4 coefficient tables
clean_terms <- function(x) {
  x %>%
    mutate(term = ifelse(term == "(Intercept)", "Intercept", term))
}

m0_coef <- tidy(m0, conf.int = TRUE) %>% clean_terms()
m1_coef <- tidy(m1, conf.int = TRUE) %>% clean_terms()
m2_coef <- tidy(m2, conf.int = TRUE) %>% clean_terms()
m3_coef <- tidy(m3, conf.int = TRUE) %>% clean_terms()

m0_coef %>%
  kable(digits = 3, caption = "Coefficient table for M0.") %>%
  kable_styling(full_width = FALSE)

m1_coef %>%
  kable(digits = 3, caption = "Coefficient table for M1.") %>%
  kable_styling(full_width = FALSE)

m2_coef %>%
  kable(digits = 3, caption = "Coefficient table for M2.") %>%
  kable_styling(full_width = FALSE)

m3_coef %>%
  kable(digits = 3, caption = "Coefficient table for M3.") %>%
  kable_styling(full_width = FALSE)

# 3.2.5 diagnostic plots
plot_model_diagnostics <- function(model, model_name, data) {
  aug <- augment(model, data = data)

  caption_text <- switch(
    model_name,
    "M0" = "Figure 6: Diagnostic plots for M0. ",
    "M1" = "Figure 7: Diagnostic plots for M1. ",
    "M2" = "Figure 8: Diagnostic plots for M2. ",
    "M3" = "Figure 9: Diagnostic plots for M3. "
  )

  p1 <- ggplot(aug, aes(x = .fitted, y = .resid)) +
    geom_point(alpha = 0.5) +
    geom_hline(yintercept = 0, linetype = "dashed") +
    geom_smooth(se = FALSE) +
    theme_bw() +
    labs(
      title = paste(model_name, ": Residuals vs fitted"),
      x = "Fitted values",
      y = "Residuals"
    )
```

```
  p2 <- ggplot(aug, aes(sample = .std.resid)) +
    stat_qq(alpha = 0.5) +
    stat_qq_line() +
    theme_bw() +
    labs(
      title = paste(model_name, ": Normal Q-Q plot"),
      x = "Theoretical quantiles",
      y = "Standardised residuals"
    )

  p3 <- ggplot(aug, aes(x = .fitted, y = sqrt(abs(.std.resid)))) +
    geom_point(alpha = 0.5) +
    geom_smooth(se = FALSE) +
    theme_bw() +
    labs(
      title = paste(model_name, ": Scale-location"),
      x = "Fitted values",
      y = expression(sqrt("|Standardised residuals|"))
    )

  p4 <- ggplot(aug, aes(x = date, y = .resid)) +
    geom_line(alpha = 0.6) +
    geom_hline(yintercept = 0, linetype = "dashed") +
    theme_bw() +
    labs(
      title = paste(model_name, ": Residuals over time"),
      x = "Date",
      y = "Residuals"
    )

  ((p1 | p2) / (p3 | p4)) +
    plot_annotation(
      caption = caption_text
    ) &
    theme(
      plot.caption = element_text(hjust = 0.5, size = 10)
    )
}

diag_m0 <- plot_model_diagnostics(m0, "M0", cycle_daily_df)
diag_m1 <- plot_model_diagnostics(m1, "M1", cycle_daily_df)
diag_m2 <- plot_model_diagnostics(m2, "M2", cycle_daily_df)
diag_m3 <- plot_model_diagnostics(m3, "M3", cycle_daily_df)

diag_m0
diag_m1
diag_m2
diag_m3

#model summary tables

glance(m0) %>%
  kable(digits = 3, caption = "Model summary statistics for M0.") %>%
```

```r
  kable_styling(full_width = FALSE)

glance(m1) %>%
  kable(digits = 3, caption = "Model summary statistics for M1.") %>%
  kable_styling(full_width = FALSE)

glance(m2) %>%
  kable(digits = 3, caption = "Model summary statistics for M2.") %>%
  kable_styling(full_width = FALSE)

glance(m3) %>%
  kable(digits = 3, caption = "Model summary statistics for M3(temperature range extension).") %>%
  kable_styling(full_width = FALSE)

# 4. Cross-Validation Functions (table 1 and 2)
#scoring function
calc_scores <- function(y, mu, sigma, alpha = 0.05) {
  # y     : vector of observed values
  # mu    : vector of predictive means (from predict(fit, newdata=test)$fit)
  # sigma : vector of predictive SDs. Combine residual error and mean uncertainty:
  #         sigma = sqrt(summary(fit)$sigma^2 + predict(fit, newdata=test, se.fit=TRUE)$se.fit^2)
  # Returns a named list with RMSE, MAE, DS, IS

  # Implement RMSE, MAE, DS, and IS:
  rmse <- sqrt(mean((y - mu)^2, na.rm = TRUE))
  mae  <- mean(abs(y - mu), na.rm = TRUE)

  ds <- mean(log(sigma^2) + ((y - mu)^2 / sigma^2), na.rm = TRUE)

  z <- qnorm(1 - alpha / 2)
  l <- mu - z * sigma
  u <- mu + z * sigma

  is <- mean(
    (u - l) +
      (2 / alpha) * pmax(l - y, 0) +
      (2 / alpha) * pmax(y - u, 0),
    na.rm = TRUE
  )

  tibble(
    RMSE = rmse,
    MAE  = mae,
    DS   = ds,
    IS   = is
  )
}
calc_scores(
  y = c(10, 12, 15),
  mu = c(11, 11, 14),
  sigma = c(2, 2, 2)
)
```

```r
# 5. Leave-One-Year-Out CV Loop
#m0 leave one year out CV
years <- sort(unique(cycle_daily_df$year))

cv_m0_list <- list()

for (yr in years) {
  train <- cycle_daily_df %>% filter(year != yr)
  test  <- cycle_daily_df %>% filter(year == yr)

  fit_m0_cv <- lm(count ~ temp_mean + weekend + month, data = train)

  pred <- predict(fit_m0_cv, newdata = test, se.fit = TRUE)
  sigma <- sqrt(summary(fit_m0_cv)$sigma^2 + pred$se.fit^2)

  fold_scores <- calc_scores(
    y = test$count,
    mu = pred$fit,
    sigma = sigma
  ) %>%
    mutate(HoldoutYear = yr)

  cv_m0_list[[as.character(yr)]] <- fold_scores
}

cv_m0_yearly <- bind_rows(cv_m0_list) %>%
  select(HoldoutYear, RMSE, MAE, DS, IS)

cv_m0_yearly

cv_m0_summary <- cv_m0_yearly %>%
  summarise(
    RMSE = mean(RMSE),
    MAE  = mean(MAE),
    DS   = mean(DS),
    IS   = mean(IS)
  )

cv_m0_summary

#m1 leave one year out CV
cv_m1_list <- list()

for (yr in years) {
  train <- cycle_daily_df %>% filter(year != yr)
  test  <- cycle_daily_df %>% filter(year == yr)

  # CV version uses numeric month instd of factor(month)
  fit_m1_cv <- lm(count ~ temp_mean + weekend + trend + month + dow_model, data = train)

  pred <- predict(fit_m1_cv, newdata = test, se.fit = TRUE)
  sigma <- sqrt(summary(fit_m1_cv)$sigma^2 + pred$se.fit^2)
```

```r
  fold_scores <- calc_scores(
    y = test$count,
    mu = pred$fit,
    sigma = sigma
  ) %>%
    mutate(HoldoutYear = yr)

  cv_m1_list[[as.character(yr)]] <- fold_scores
}

cv_m1_yearly <- bind_rows(cv_m1_list) %>%
  select(HoldoutYear, RMSE, MAE, DS, IS)

cv_m1_yearly

cv_m1_summary <- cv_m1_yearly %>%
  summarise(
    RMSE = mean(RMSE),
    MAE  = mean(MAE),
    DS   = mean(DS),
    IS   = mean(IS)
  )

cv_m1_summary

#m2 leave one year out CV
cv_m2_list <- list()

for (yr in years) {
  train <- cycle_daily_df %>% filter(year != yr)
  test  <- cycle_daily_df %>% filter(year == yr)

  fit_m2_cv <- lm(count ~ temp_mean + I(temp_mean^2) + weekend + trend + month + dow_model,
                  data = train)

  pred <- predict(fit_m2_cv, newdata = test, se.fit = TRUE)
  sigma <- sqrt(summary(fit_m2_cv)$sigma^2 + pred$se.fit^2)

  fold_scores <- calc_scores(
    y = test$count,
    mu = pred$fit,
    sigma = sigma
  ) %>%
    mutate(HoldoutYear = yr)

  cv_m2_list[[as.character(yr)]] <- fold_scores
}

cv_m2_yearly <- bind_rows(cv_m2_list) %>%
  select(HoldoutYear, RMSE, MAE, DS, IS)

cv_m2_yearly
```

```r
cv_m2_summary <- cv_m2_yearly %>%
  summarise(
    RMSE = mean(RMSE),
    MAE  = mean(MAE),
    DS   = mean(DS),
    IS   = mean(IS)
  )

cv_m2_summary

#m3 leave one year out CV
cv_m3_list <- list()

for (yr in years) {
  train <- cycle_daily_df %>% filter(year != yr)
  test  <- cycle_daily_df %>% filter(year == yr)

  fit <- lm(count ~ temp_mean + I(temp_mean^2) + weekend + trend + month + dow_model + temp_range,
            data = train)

  pred <- predict(fit, newdata = test, se.fit = TRUE)
  sigma <- sqrt(summary(fit)$sigma^2 + pred$se.fit^2)

  cv_m3_list[[as.character(yr)]] <- calc_scores(
    y = test$count,
    mu = pred$fit,
    sigma = sigma
  ) %>%
    mutate(HoldoutYear = yr)
}

cv_m3_yearly <- bind_rows(cv_m3_list)

cv_m3_summary <- cv_m3_yearly %>%
  summarise(
    RMSE = mean(RMSE),
    MAE  = mean(MAE),
    DS   = mean(DS),
    IS   = mean(IS)
  )

cv_m3_yearly
cv_m3_summary

# table 1 cv scores for m0 to m3
table1_cv <- bind_rows(
  cv_m0_summary %>% mutate(Model = "M0"),
  cv_m1_summary %>% mutate(Model = "M1"),
  cv_m2_summary %>% mutate(Model = "M2"),
  cv_m3_summary %>% mutate(Model = "M3"),
) %>%
  select(Model, RMSE, MAE, DS, IS)
```

```r
table1_cv

table1_cv %>%
  kable(
    digits = c(0, 1, 1, 2, 1),
    caption = "Leave-one-year-out CV scores for M0-M3. Lower is better for all scores."
  ) %>%
  kable_styling(full_width = FALSE)

# 6. CV by Month
months <- 1:12
cv_month_list <- list()

for (m in months) {
  train <- cycle_daily_df %>% filter(month != m)
  test  <- cycle_daily_df %>% filter(month == m)

  fit_final_cv <- lm(
    count ~ temp_mean + I(temp_mean^2) + weekend + trend + month + dow_model,
    data = train
  )

  pred <- predict(fit_final_cv, newdata = test, se.fit = TRUE)
  sigma <- sqrt(summary(fit_final_cv)$sigma^2 + pred$se.fit^2)

  fold_scores <- calc_scores(
    y = test$count,
    mu = pred$fit,
    sigma = sigma
  ) %>%
    mutate(MonthNum = m)

  cv_month_list[[as.character(m)]] <- fold_scores
}

table2_month_cv <- bind_rows(cv_month_list) %>%
  mutate(
    Month = factor(MonthNum, levels = 1:12, labels = month.abb, ordered = TRUE)
  ) %>%
  select(Month, RMSE, DS) %>%
  arrange(Month)

table2_month_cv %>%
  kable(
    digits = c(0, 1, 2),
    caption = "CV RMSE and DS by month for the final model (M2)."
  ) %>%
  kable_styling(full_width = FALSE)

# Q5B practical performance
#5.1 temperature effects and nonlinearity
#fit M2 three times using temp_mean, temp_min, temp_max
temp_specs_m2 <- list(
```

```
  temp_mean = count ~ temp_mean + I(temp_mean^2) + weekend + trend + month + dow_model,
  temp_min  = count ~ temp_min  + I(temp_min^2)  + weekend + trend + month + dow_model,
  temp_max  = count ~ temp_max  + I(temp_max^2)  + weekend + trend + month + dow_model
)

temp_cv_results <- vector("list", length(temp_specs_m2))
names(temp_cv_results) <- names(temp_specs_m2)

for (temp_name in names(temp_specs_m2)) {
  fold_list <- vector("list", length(years))
  names(fold_list) <- as.character(years)

  for (yr in years) {
    train <- cycle_daily_df %>% filter(year != yr)
    test  <- cycle_daily_df %>% filter(year == yr)

    fit <- lm(temp_specs_m2[[temp_name]], data = train)
    pred <- predict(fit, newdata = test, se.fit = TRUE)
    sigma <- sqrt(summary(fit)$sigma^2 + pred$se.fit^2)

    fold_list[[as.character(yr)]] <- calc_scores(
      y = test$count,
      mu = pred$fit,
      sigma = sigma
    )
  }

  temp_cv_results[[temp_name]] <- bind_rows(fold_list) %>%
    summarise(
      RMSE = mean(RMSE),
      DS   = mean(DS),
      IS   = mean(IS)
    ) %>%
    mutate(Temperature = temp_name)
}

table_temp_compare <- bind_rows(temp_cv_results) %>%
  select(Temperature, RMSE, DS, IS)

table_temp_compare %>%
  kable(
    digits = c(0, 1, 2, 1),
    caption = "CV comparison of M2 fitted with alternative temperature variables."
  ) %>%
  kable_styling(full_width = FALSE)

table_temp_compare

# base on cv table temp_max gives the best predictive performance
m2_tempmax <- lm(
  count ~ temp_max + I(temp_max^2) + weekend + trend + factor(month) + dow_model,
  data = cycle_daily_df
)
```

```r
#marginal effect
b1 <- unname(coef(m2_tempmax)["temp_max"])
b2 <- unname(coef(m2_tempmax)["I(temp_max^2)"])

marginal_5  <- b1 + 2 * b2 * 5
marginal_15 <- b1 + 2 * b2 * 15

table_temp_marginal <- tibble(
  Evaluation_point = c("5\u00B0C", "15\u00B0C"),
  marginal_effect = c(marginal_5, marginal_15)
)

table_temp_marginal %>%
  kable(
    digits = c(0, 1),
    col.names = c("Evaluation point", "Marginal effect (cyclists per 1\u00B0C)"),
    caption = "Estimated marginal temperature effect for the selected M2 variant."
  ) %>%
  kable_styling(full_width = FALSE)
# 5.3 longterm trend and extrapolation
#table 3 trend coefficients
get_trend_row <- function(model, model_name) {
  daily_change <- unname(coef(model)["trend"])

  tibble(
    Model = model_name,
    `Daily change (cyclists/day)` = daily_change,
    `Annual change (cyclists/year)` = daily_change * 365
  )
}

table3_trend <- bind_rows(
  get_trend_row(m1, "M1"),
  get_trend_row(m2, "M2"),
  get_trend_row(m3, "M3")
)

table3_trend %>%
  kable(
    digits = c(0, 3, 0),
    caption = "Table 3: Trend coefficients from M1-M3."
  ) %>%
  kable_styling(full_width = FALSE)

#trend significance for final model
trend_sig_m2 <- tidy(m2, conf.int = TRUE) %>%
  filter(term == "trend") %>%
  transmute(
    term,
    estimate,
    std.error,
    statistic,
    p.value,
```

```
    conf.low,
    conf.high
  )

trend_sig_m2 %>%
  kable(
    digits = 4,
    caption = "Trend inference for the final model."
  ) %>%
  kable_styling(full_width = FALSE)

#annual change as % of predicted count at start of observation period
start_date_data <- cycle_daily_df %>%
  filter(date == as.Date("2020-01-01")) %>%
  slice(1)

baseline_2020 <- tibble(
  date = as.Date("2020-01-01"),
  year = 2020,
  month = start_date_data$month,
  weekend = start_date_data$weekend,
  dow_model = factor(as.character(start_date_data$dow_model),
                     levels = levels(cycle_daily_df$dow_model)),
  trend = 0,
  temp_mean = start_date_data$temp_mean
)

start_pred <- predict(m2, newdata = baseline_2020)

annual_change_m2 <- unname(coef(m2)["trend"]) * 365
annual_pct_m2 <- 100 * annual_change_m2 / start_pred

annual_change_summary <- tibble(
  `Predicted count at 2020-01-01` = start_pred,
  `Annual change (cyclists/year)` = annual_change_m2,
  `Annual percentage change` = annual_pct_m2
)

annual_change_summary %>%
  kable(
    digits = c(1, 0, 2),
    caption = "Annual change implied by the final model, relative to 1 January 2020."
  ) %>%
  kable_styling(full_width = FALSE)

# compare m1 m2 m3 trend estimates
trend_consistency <- bind_rows(
  tidy(m1, conf.int = TRUE) %>% filter(term == "trend") %>% mutate(Model = "M1"),
  tidy(m2, conf.int = TRUE) %>% filter(term == "trend") %>% mutate(Model = "M2"),
  tidy(m3, conf.int = TRUE) %>% filter(term == "trend") %>% mutate(Model = "M3")
) %>%
  select(Model, estimate, std.error, conf.low, conf.high, p.value)
```

```r
trend_consistency %>%
  kable(
    digits = 4,
    caption = "Comparison of trend estimates across M1, M2, and M3."
  ) %>%
  kable_styling(full_width = FALSE)

#projection to 2035 under fixed summer weekday conditions
projection_df <- tibble(
  date = seq(as.Date("2020-01-01"), as.Date("2035-12-31"), by = "day")
) %>%
  mutate(
    year = year(date),
    month = 7,
    weekend = 0,
    dow_model = factor("Wed", levels = levels(cycle_daily_df$dow_model)),
    trend = as.integer(date - as.Date("2020-01-01")),
    temp_mean = mean(cycle_daily_df$temp_mean[cycle_daily_df$month %in% c(6, 7, 8)], na.rm = TRUE)
  )

proj_pred <- predict(m2, newdata = projection_df, se.fit = TRUE)

projection_df <- projection_df %>%
  mutate(
    fit = proj_pred$fit,
    lwr = fit - 1.96 * sqrt(summary(m2)$sigma^2 + proj_pred$se.fit^2),
    upr = fit + 1.96 * sqrt(summary(m2)$sigma^2 + proj_pred$se.fit^2)
  )

p_projection <- ggplot(projection_df, aes(x = date, y = fit)) +
  geom_line() +
  geom_ribbon(aes(ymin = lwr, ymax = upr), alpha = 0.2) +
  theme_bw() +
  labs(
    title = "Figure 10: Projected daily cycling demand to 2035",
    x = "Date",
    y = "Predicted daily cyclist count"
  )

p_projection

#thresholds: here we do two thresholds, 15000 as hinted in the instructions gives a rather early thresh
threshold <- 15000

threshold_hits <- projection_df %>%
  filter(fit <= threshold)

threshold_date <- if (nrow(threshold_hits) == 0) {
  NA
} else {
  min(threshold_hits$date)
}
```

```
threshold_date

threshold <- 10000

threshold_hits <- projection_df %>%
  filter(fit <= threshold)

threshold_date <- if (nrow(threshold_hits) == 0) {
  NA
} else {
  min(threshold_hits$date)
}

threshold_date
# 5.4 identify poorly fit period
#locate worst month/period from residuals
aug_m2 <- augment(m2, data = cycle_daily_df)

monthly_resid_summary <- aug_m2 %>%
  mutate(year_month = format(date, "%Y-%m")) %>%
  group_by(year_month) %>%
  summarise(
    mean_residual = mean(.resid, na.rm = TRUE),
    rmse = sqrt(mean(.resid^2, na.rm = TRUE)),
    .groups = "drop"
  ) %>%
  arrange(desc(rmse))

monthly_resid_summary %>%
  kable(
    digits = c(0, 1, 1),
    caption = "Monthly residual summary for the final model (M2), ordered by RMSE."
  ) %>%
  kable_styling(full_width = FALSE)

# pick the worst fit month
worst_period <- monthly_resid_summary %>% slice(1)
worst_period_label <- worst_period$year_month

worst_start <- as.Date(paste0(worst_period_label, "-01"))
worst_end <- ceiling_date(worst_start, unit = "month") - days(1)

worst_period_stats <- aug_m2 %>%
  filter(date >= worst_start, date <= worst_end) %>%
  summarise(
    mean_residual = mean(.resid, na.rm = TRUE),
    rmse = sqrt(mean(.resid^2, na.rm = TRUE))
  ) %>%
  mutate(
    period = worst_period_label,
    direction = case_when(
      mean_residual > 0 ~ "Under-prediction on average",
      mean_residual < 0 ~ "Over-prediction on average",
```

```
      TRUE ~ "No systematic bias on average"
    )
  ) %>%
  select(period, mean_residual, rmse, direction)

worst_period_stats %>%
  kable(
    digits = c(0, 1, 1, 0),
    caption = "Residual summary for the single worst fit period identified from the final model (M2)."
  ) %>%
  kable_styling(full_width = FALSE)

p_resid_time <- ggplot(aug_m2, aes(x = date, y = .resid)) +
  geom_line(alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_rect(
    aes(
      xmin = worst_start,
      xmax = worst_end,
      ymin = -Inf,
      ymax = Inf
    ),
    inherit.aes = FALSE,
    alpha = 0.2
  ) +
  theme_bw() +
  labs(
    title = "Figure 11: Residuals over time for the final model (M2)",
    x = "Date",
    y = "Residual",
    caption = paste(
      "Residuals over time for M2,",
      worst_period_label,
      "highlighted as the worst fit month. Positive residuals indicate under prediction."
    )
  )

p_resid_time

# 5.5 Original diagnostic / insight plot
# residuals heatmap to inspect bias
resid_heatmap_df <- aug_m2 %>%
  mutate(
    Year = factor(year(date)),
    Month = factor(month(date), levels = 1:12, labels = month.abb, ordered = TRUE)
  ) %>%
  group_by(Year, Month) %>%
  summarise(
    mean_residual = mean(.resid, na.rm = TRUE),
    .groups = "drop"
  )

p_resid_heatmap <- ggplot(resid_heatmap_df, aes(x = Month, y = Year, fill = mean_residual)) +
```

```
  geom_tile(color = "white") +
  geom_text(aes(label = round(mean_residual, 0)), size = 3) +
  theme_bw() +
  labs(
    title = "Figure 12: Systematic prediction bias by month and year for the final model (M2)",
    x = "Month",
    y = "Year",
    fill = "Mean residual",
    caption = "Mean residuals by month and year for the final model. Forecast bias is not uniform over
  )

p_resid_heatmap
```